## Q1. Build a Basic Autoencoder

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader
import torchvision.datasets as datasets
import torchvision.transforms as transforms
import numpy as np
import matplotlib.pyplot as plt
```

```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
```

```
print(f"Using device: {device}")
print("Core libraries for Autoencoder assignment imported.")
```

```
Using device: cuda
Core libraries for Autoencoder assignment imported.
```

```
latent_dim_q1 = 64  # Latent space dimension for this simple AutoEncoder
batch_size_q1 = 128
epochs_q1 = 10 # doing 10 epochs for a quick run
learning_rate_q1 = 0.001
```

```
# Loading the MNIST dataset
print("Loading and normalizing MNIST dataset...")
# Transform to tensor and normalize to [0,1]
transform_q1 = transforms.Compose([
    transforms.ToTensor(),
])
```

```
Loading and normalizing MNIST dataset...
```

```
train_dataset_q1 = datasets.MNIST(root='./data_mnist_ae', train=True, transform=transform_q1, download=True)
# I will use the train_loader for training and also for visualization later
train_loader_q1 = DataLoader(dataset=train_dataset_q1, batch_size=batch_size_q1, shuffle=True)
print("MNIST dataset loaded.")
```

```
100%|██████████| 9.91M/9.91M [00:00<00:00, 16.3MB/s]
100%|██████████| 28.9k/28.9k [00:00<00:00, 481kB/s]
100%|██████████| 1.65M/1.65M [00:00<00:00, 3.80MB/s]
100%|██████████| 4.54k/4.54k [00:00<00:00, 4.07MB/s]MNIST dataset loaded.
```

```
# 2) Build a simple autoencoder using Dense layers.
class BasicAutoencoder(nn.Module):
    def __init__(self, input_dim=28*28, latent_dim=latent_dim_q1):
        super(BasicAutoencoder, self).__init__()
        # Encoder
        self.encoder = nn.Sequential(
            nn.Linear(input_dim, 256),
            nn.ReLU(),
            nn.Linear(256, 128),
            nn.ReLU(),
            nn.Linear(128, latent_dim),
            nn.ReLU() #
        )
        # Decoder
        self.decoder = nn.Sequential(
            nn.Linear(latent_dim, 128),
            nn.ReLU(),
            nn.Linear(128, 256),
            nn.ReLU(),
            nn.Linear(256, input_dim),
            nn.Sigmoid()
        )
    def forward(self, x):
        x_flat = x.view(x.size(0), -1)
        encoded = self.encoder(x_flat)
        decoded = self.decoder(encoded)
        reconstructed_x = decoded.view(x.size(0), 1, 28, 28)
        return reconstructed_x, encoded
```

```
model_q1 = BasicAutoencoder().to(device)
print("\n Simple Autoencoder model structure:")
print(model_q1)
```

```

 Simple Autoencoder model structure:
BasicAutoencoder(
  (encoder): Sequential(
```

```
    (0): Linear(in_features=784, out_features=256, bias=True)
    (1): ReLU()
    (2): Linear(in_features=256, out_features=128, bias=True)
    (3): ReLU()
    (4): Linear(in_features=128, out_features=64, bias=True)
    (5): ReLU()
  )
  (decoder): Sequential(
    (0): Linear(in_features=64, out_features=128, bias=True)
    (1): ReLU()
    (2): Linear(in_features=128, out_features=256, bias=True)
    (3): ReLU()
    (4): Linear(in_features=256, out_features=784, bias=True)
    (5): Sigmoid()
  )
)
```

```
def train_basic_autoencoder(model, dataloader, epochs, lr):
    model.train() # Set model to training mode
    optimizer = optim.Adam(model.parameters(), lr=lr)
    criterion = nn.MSELoss() # Mean Squared Error loss is common for reconstruction

    print(f"\nStarting training for {epochs} epochs...")
    for epoch in range(epochs):
        total_epoch_loss = 0
        for batch_images, _ in dataloader:
            batch_images = batch_images.to(device)

            optimizer.zero_grad()
            reconstructed_images, _ = model(batch_images) # We only need reconstructed for loss
            loss = criterion(reconstructed_images, batch_images) # ComparING reconstruction to original
            loss.backward()
            optimizer.step()
            total_epoch_loss += loss.item() * batch_images.size(0)

        avg_epoch_loss = total_epoch_loss / len(dataloader.dataset)
        print(f"Epoch [{epoch+1}/{epochs}], Average Loss: {avg_epoch_loss:.4f}")
    print("Training complete.")
```

```
train_basic_autoencoder(model_q1, train_loader_q1, epochs=epochs_q1, lr=learning_rate_q1)
```

```
Starting training for 10 epochs...
Epoch [1/10], Average Loss: 0.0532
Epoch [2/10], Average Loss: 0.0267
Epoch [3/10], Average Loss: 0.0207
Epoch [4/10], Average Loss: 0.0171
Epoch [5/10], Average Loss: 0.0151
Epoch [6/10], Average Loss: 0.0135
Epoch [7/10], Average Loss: 0.0123
Epoch [8/10], Average Loss: 0.0114
Epoch [9/10], Average Loss: 0.0106
Epoch [10/10], Average Loss: 0.0100
Training complete.
```

```
# Display 5 original and reconstructed images side by side.
print("\nDisplaying original vs. reconstructed images...")
model_q1.eval()
sample_images, _ = next(iter(train_loader_q1))
sample_images = sample_images.to(device)
```

```
Displaying original vs. reconstructed images...
```

```
with torch.no_grad():
    reconstructed_sample_images, _ = model_q1(sample_images)
```

```
sample_images_np = sample_images.cpu().numpy()
reconstructed_sample_images_np = reconstructed_sample_images.cpu().numpy()
```

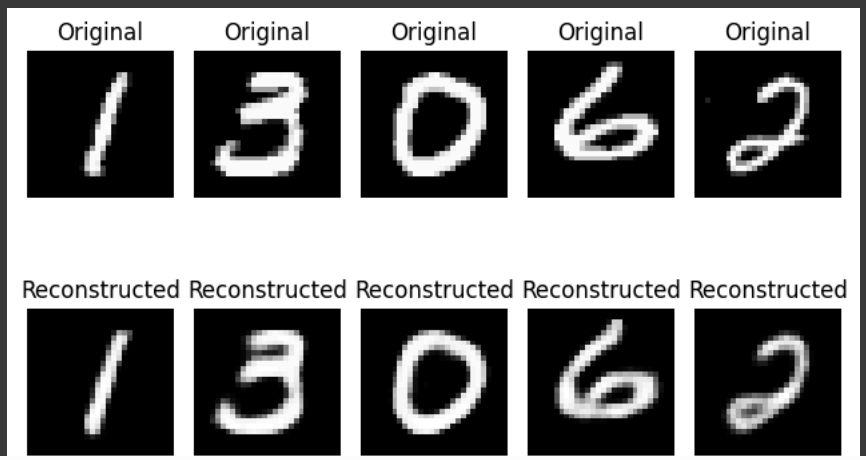
```
n_display = 5
plt.figure(figsize=(10, 4))
plt.suptitle("Original vs. Reconstructed Images (Basic Autoencoder)", fontsize=14)
```

```
Text(0.5, 0.98, 'Original vs. Reconstructed Images (Basic Autoencoder)')
<Figure size 1000x400 with 0 Axes>
```

```
for i in range(n_display):
    ax = plt.subplot(2, n_display, i + 1)
    plt.imshow(sample_images_np[i].reshape(28, 28), cmap='gray')
    ax.set_title("Original")
    ax.axis('off')

    ax = plt.subplot(2, n_display, i + 1 + n_display)
    plt.imshow(reconstructed_sample_images_np[i].reshape(28, 28), cmap='gray')
    ax.set_title("Reconstructed")
```

```
    ax.axis('off')
plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.show()
```




## Q2. Visualize Encoded Representations

```
from sklearn.manifold import TSNE
```

```
# Loading the MNIST test dataset
print("Loading MNIST test dataset...")
test_dataset_q2 = datasets.MNIST(root='./data_mnist_ae_test', train=False, transform=transform_q1, download=True)
```

```
Loading MNIST test dataset...
100%|██████████| 9.91M/9.91M [00:00<00:00, 16.2MB/s]
100%|██████████| 28.9k/28.9k [00:00<00:00, 481kB/s]
100%|██████████| 1.65M/1.65M [00:00<00:00, 3.82MB/s]
100%|██████████| 4.54k/4.54k [00:00<00:00, 5.51MB/s]
```

```
test_loader_q2 = DataLoader(dataset=test_dataset_q2, batch_size=128, shuffle=False) # batch_size can be larger
print("MNIST test dataset loaded.")
```

```
MNIST test dataset loaded.
```

```
# Extract the first 100 test images and their labels
num_samples_to_visualize = 100
all_test_images = []
all_test_labels = []
```

```
for images, labels in test_loader_q2:
    all_test_images.append(images)
    all_test_labels.append(labels)
    if len(all_test_images) * test_loader_q2.batch_size >= num_samples_to_visualize:
        break
```

```
all_test_images = torch.cat(all_test_images, dim=0)[:num_samples_to_visualize]
all_test_labels = torch.cat(all_test_labels, dim=0)[:num_samples_to_visualize]
```

```
print(f"Extracted {all_test_images.shape[0]} samples for visualization.")
```

```
Extracted 100 samples for visualization.
```

```
# Moving images to the configured device
all_test_images = all_test_images.to(device)
```

```
# Using the encoder to extract compressed features
model_q1.eval() # Setting the autoencoder model to evaluation mode
latent_features_list = []
```

```
with torch.no_grad():
  flattened_images = all_test_images.view(all_test_images.size(0), -1)
  latent_features = model_q1.encoder(flattened_images)
```

```
latent_features_np = latent_features.cpu().numpy()
labels_np = all_test_labels.cpu().numpy()
```

```
print(f"Shape of extracted latent features: {latent_features_np.shape}")
```

```
Shape of extracted latent features: (100, 64)
```

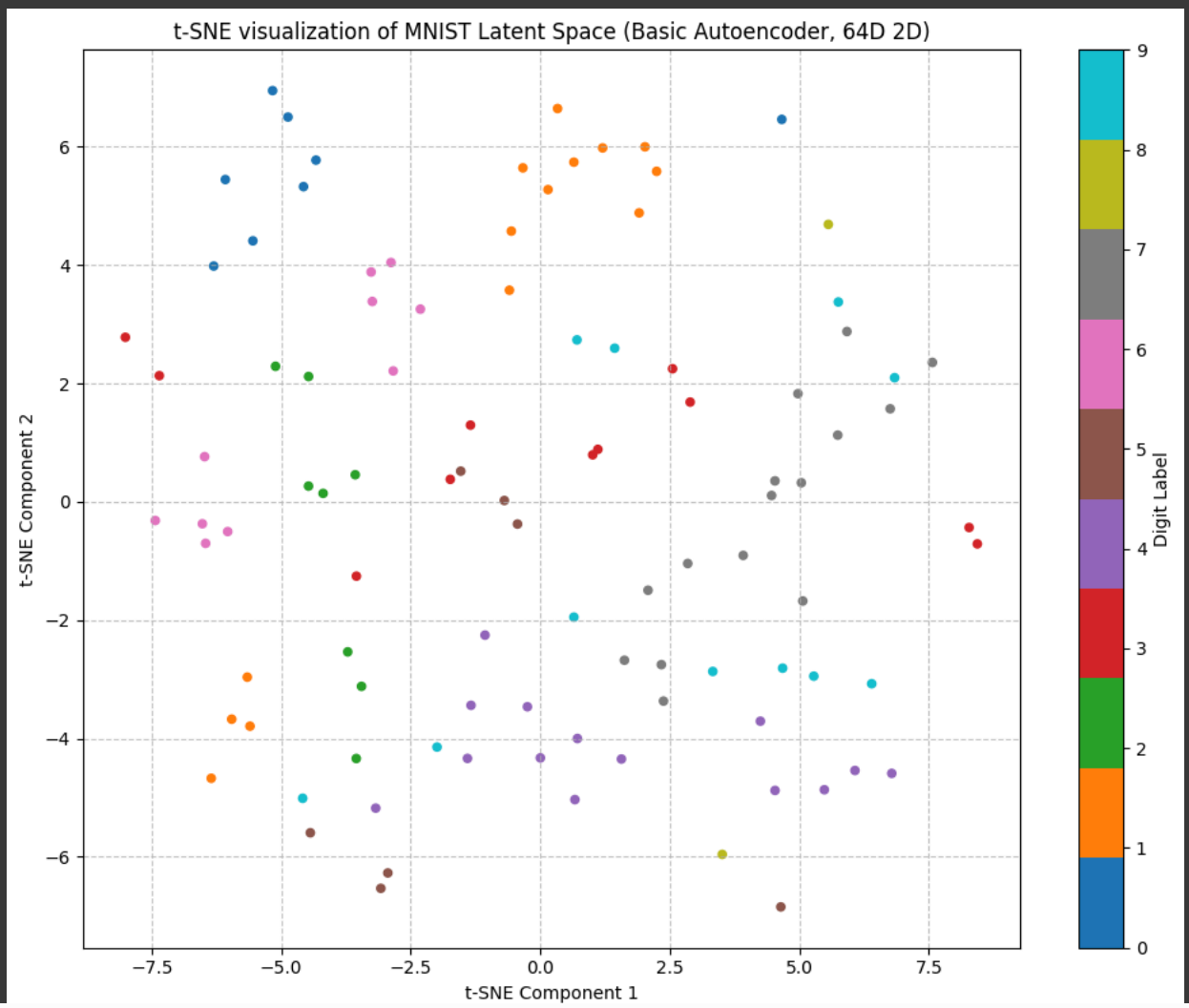
```
# Plotting the encoded features in 2D using t-SNE.
print("\nApplying t-SNE to reduce latent features to 2D...")
tsne = TSNE(n_components=2, random_state=42, perplexity=30, n_iter=300)
```

```
Applying t-SNE to reduce latent features to 2D...
```

```
latent_features_2d = tsne.fit_transform(latent_features_np)
print("t-SNE transformation complete.")
print(f"Shape of 2D latent features: {latent_features_2d.shape}")
```

```
/usr/local/lib/python3.11/dist-packages/sklearn/manifold/_t_sne.py:1164: FutureWarning: 'n_iter' was renamed to 'max_iter' in version 1.5 and will
  warnings.warn(
t-SNE transformation complete.
Shape of 2D latent features: (100, 2)
```

```
# Color the points by their digit labels.
plt.figure(figsize=(10, 8))
scatter = plt.scatter(latent_features_2d[:, 0], latent_features_2d[:, 1], c=labels_np, cmap='tab10', s=20)
plt.colorbar(scatter, ticks=range(10), label='Digit Label')
# Check if latent_dim_q1 exists, otherwise use a placeholder for the title
title_latent_dim = latent_dim_q1 if 'latent_dim_q1' in locals() else 'N'
plt.title(f't-SNE visualization of MNIST Latent Space (Basic Autoencoder, {title_latent_dim}D 2D)')
plt.xlabel('t-SNE Component 1')
plt.ylabel('t-SNE Component 2')
plt.grid(True, linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()
```




```
import torch.nn.functional as F # For the loss function
```

## Q3. Build a Denoising Autoencoder

```
batch_size_q3 = 100
epochs_q3 = 10      # Using 10 epochs
learning_rate_q3 = 0.001
```

```
#Loading MNIST dataset
print("Loading MNIST dataset...")
```

```
transform_q3 = transforms.ToTensor()
```

```
Loading MNIST dataset...
```

```
train_dataset_q3 = datasets.MNIST(root='./data_mnist_denoise', train=True, transform=transform_q3, download=True)
test_dataset_q3 = datasets.MNIST(root='./data_mnist_denoise', train=False, transform=transform_q3, download=True)
```

```
100%|██████████| 9.91M/9.91M [00:00<00:00, 17.6MB/s]
100%|██████████| 28.9k/28.9k [00:00<00:00, 482kB/s]
100%|██████████| 1.65M/1.65M [00:00<00:00, 3.29MB/s]
100%|██████████| 4.54k/4.54k [00:00<00:00, 13.9MB/s]
```

```
train_loader_q3 = DataLoader(dataset=train_dataset_q3, batch_size=batch_size_q3, shuffle=True)
test_loader_q3 = DataLoader(dataset=test_dataset_q3, batch_size=batch_size_q3, shuffle=False)
print("MNIST dataset loaded.")
```

```
MNIST dataset loaded.
```

```
# Add random noise to the MNIST images.
def add_noise(images, noise_factor=0.5):
    noise = torch.randn_like(images) * noise_factor
    noisy_images = images + noise
    noisy_images = torch.clamp(noisy_images, 0., 1.) # Keep pixel values in [0,1]
    return noisy_images
```

```
# Define a Simple Convolutional Autoencoder
class SimpleConvAE_Denoise(nn.Module):
    def __init__(self):
        super(SimpleConvAE_Denoise, self).__init__()
        # Encoder
        self.encoder = nn.Sequential(
            nn.Conv2d(1, 16, kernel_size=3, stride=2, padding=1),
            nn.ReLU(),
            nn.Conv2d(16, 32, kernel_size=3, stride=2, padding=1),
            nn.ReLU(),
            nn.Flatten(),
            nn.Linear(32*7*7, 128),
            nn.ReLU()
        )
        # Decoder
        self.decoder = nn.Sequential(
            nn.Linear(128, 32*7*7),
            nn.ReLU(), # Added ReLU here for symmetry
            nn.Unflatten(1, (32, 7, 7)),
            nn.ConvTranspose2d(32, 16, kernel_size=3, stride=2, padding=1, output_padding=1),
            nn.ReLU(),
            nn.ConvTranspose2d(16, 1, kernel_size=3, stride=2, padding=1, output_padding=1),
            nn.Sigmoid()
        )

    def forward(self, x):
        code = self.encoder(x)
        reconstructed = self.decoder(code)
        return reconstructed, code
```

```
def bce_loss_fn(recon_x, x):
    return F.binary_cross_entropy(recon_x, x, reduction='sum')
```

```
model_denoise_q3 = SimpleConvAE_Denoise().to(device)
optimizer_q3 = optim.Adam(model_denoise_q3.parameters(), lr=learning_rate_q3)
print("\nDenoising Autoencoder model structure:")
print(model_denoise_q3)
```

```

Denoising Autoencoder model structure:
SimpleConvAE_Denoise(
  (encoder): Sequential(
    (0): Conv2d(1, 16, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1))
    (1): ReLU()
    (2): Conv2d(16, 32, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1))
    (3): ReLU()
    (4): Flatten(start_dim=1, end_dim=-1)
    (5): Linear(in_features=1568, out_features=128, bias=True)
    (6): ReLU()
  )
  (decoder): Sequential(
    (0): Linear(in_features=128, out_features=1568, bias=True)
    (1): ReLU()
    (2): Unflatten(dim=1, unflattened_size=(32, 7, 7))
    (3): ConvTranspose2d(32, 16, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), output_padding=(1, 1))
    (4): ReLU()
    (5): ConvTranspose2d(16, 1, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), output_padding=(1, 1))
    (6): Sigmoid()
  )
)
```

```
# Train an autoencoder to take noisy images as input and clean images as output.
print("\nTraining the Denoising Autoencoder...")
model_denoise_q3.train() # Setting model to training mode
```

```
Training the Denoising Autoencoder...
SimpleConvAE_Denoise(
  (encoder): Sequential(
    (0): Conv2d(1, 16, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1))
    (1): ReLU()
    (2): Conv2d(16, 32, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1))
    (3): ReLU()
    (4): Flatten(start_dim=1, end_dim=-1)
    (5): Linear(in_features=1568, out_features=128, bias=True)
    (6): ReLU()
  )
  (decoder): Sequential(
    (0): Linear(in_features=128, out_features=1568, bias=True)
    (1): ReLU()
    (2): Unflatten(dim=1, unflattened_size=(32, 7, 7))
    (3): ConvTranspose2d(32, 16, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), output_padding=(1, 1))
    (4): ReLU()
    (5): ConvTranspose2d(16, 1, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), output_padding=(1, 1))
    (6): Sigmoid()
  )
)
```

```
for epoch in range(epochs_q3):
    total_epoch_loss = 0
    for clean_images, _ in train_loader_q3:
        clean_images = clean_images.to(device)
        noisy_images = add_noise(clean_images).to(device) # Creating noisy versions for input

        optimizer_q3.zero_grad()
        reconstructed_images, _ = model_denoise_q3(noisy_images) # Model sees noisy, tries to output clean

        loss = bce_loss_fn(reconstructed_images, clean_images) # Compare reconstruction to ORIGINAL CLEAN images
        loss.backward()
        optimizer_q3.step()

        total_epoch_loss += loss.item()

    # Average loss per sample for the epoch
    avg_epoch_loss = total_epoch_loss / len(train_loader_q3.dataset)
    print(f"Epoch [{epoch+1}/{epochs_q3}], Average Loss: {avg_epoch_loss:.4f}")

print("Training finished!")
```

```
Epoch [1/10], Average Loss: 176.8766
Epoch [2/10], Average Loss: 96.4711
Epoch [3/10], Average Loss: 87.4807
Epoch [4/10], Average Loss: 84.0259
Epoch [5/10], Average Loss: 82.1399
Epoch [6/10], Average Loss: 80.9510
Epoch [7/10], Average Loss: 80.1297
Epoch [8/10], Average Loss: 79.5131
Epoch [9/10], Average Loss: 78.9538
Epoch [10/10], Average Loss: 78.5452
Training finished!
```

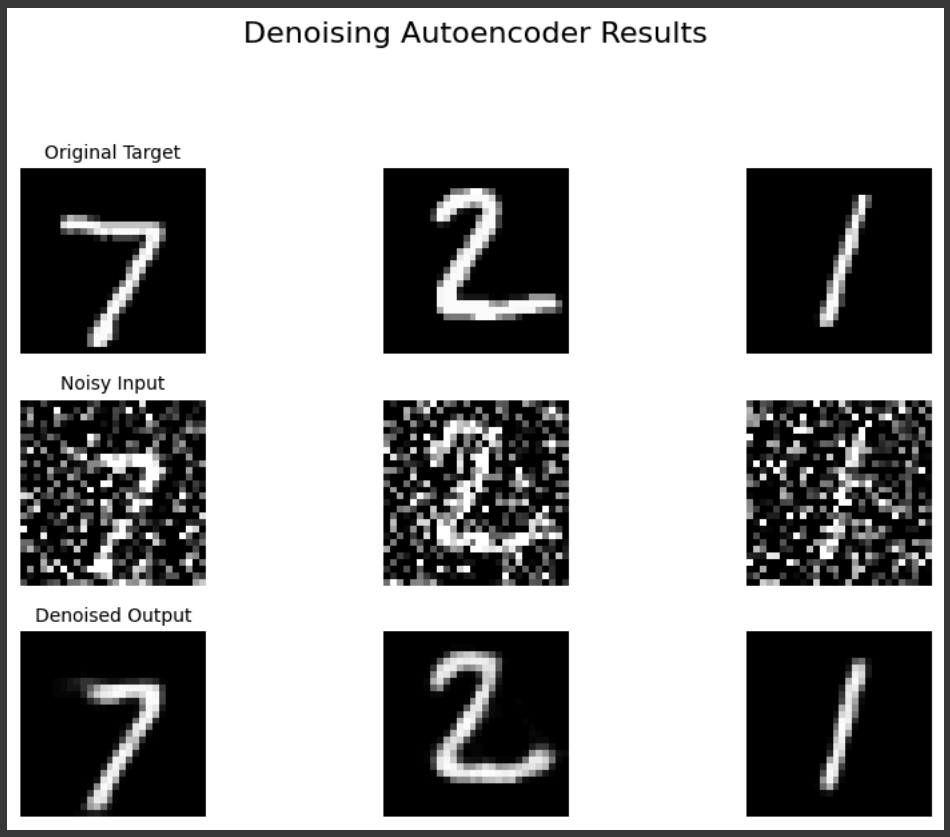
```
# Showing 3 examples of (Noisy Input → Denoised Output → Original Target).
print("\nDisplaying Denoising Examples...")
model_denoise_q3.eval() # Set model to evaluation mode
```

```
Displaying Denoising Examples...
SimpleConvAE_Denoise(
  (encoder): Sequential(
    (0): Conv2d(1, 16, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1))
    (1): ReLU()
    (2): Conv2d(16, 32, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1))
    (3): ReLU()
    (4): Flatten(start_dim=1, end_dim=-1)
    (5): Linear(in_features=1568, out_features=128, bias=True)
    (6): ReLU()
  )
  (decoder): Sequential(
    (0): Linear(in_features=128, out_features=1568, bias=True)
    (1): ReLU()
    (2): Unflatten(dim=1, unflattened_size=(32, 7, 7))
    (3): ConvTranspose2d(32, 16, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), output_padding=(1, 1))
    (4): ReLU()
    (5): ConvTranspose2d(16, 1, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), output_padding=(1, 1))
    (6): Sigmoid()
  )
)
```

```
# Getting a batch of images from the test_loader for visualization
original_test_images, _ = next(iter(test_loader_q3))
original_test_images = original_test_images.to(device)
```

```
# Creating noisy versions of these test images
noisy_test_images = add_noise(original_test_images).to(device)
```

```
with torch.no_grad():
    denoised_test_images, _ = model_denoise_q3(noisy_test_images)
```

```
original_test_images_np = original_test_images.cpu().numpy()
noisy_test_images_np = noisy_test_images.cpu().numpy()
denoised_test_images_np = denoised_test_images.cpu().numpy()
```

```
n_display = 3
plt.figure(figsize=(9, 6)) # figsize=(width, height)
plt.suptitle("Denoising Autoencoder Results", fontsize=16, y=1.02)
for i in range(n_display):
    # Display Original Image
    ax = plt.subplot(3, n_display, i + 1) # 3 rows, n_display columns
    plt.imshow(original_test_images_np[i].reshape(28, 28), cmap='gray')
    if i == 0: ax.set_title("Original Target", fontsize=10)
    ax.axis('off')

    # Display Noisy Input
    ax = plt.subplot(3, n_display, i + 1 + n_display)
    plt.imshow(noisy_test_images_np[i].reshape(28, 28), cmap='gray')
    if i == 0: ax.set_title("Noisy Input", fontsize=10)
    ax.axis('off')

    # Display Denoised Output
    ax = plt.subplot(3, n_display, i + 1 + 2*n_display)
    plt.imshow(denoised_test_images_np[i].reshape(28, 28), cmap='gray')
    if i == 0: ax.set_title("Denoised Output", fontsize=10)
    ax.axis('off')

plt.tight_layout(rect=[0, 0, 1, 0.95])
plt.show()
```